

NAME-SAHAJPREET SINGH

SRN-PES1UG24CS397

SECTION-G

Lab 4 – Vibe Coding | Deliverable

1) Before Video Link

<https://drive.google.com/drive/folders/1-MdHF8xRGwgp5glnLVj5b4kl2mtdsWeS?usp=sharing>

2) After Video Link

<https://drive.google.com/drive/folders/1hHGn3mVHlQ2PF67Ik46YlLc5dDzJLjfb?usp=sharing>

3) Prompt Evidence Link/ Chat History Make sure to make it public

<https://claude.ai/share/9d98d40f-40de-42c1-949f-ca1ca95eb581>

4) Features/Bug Fixed Snippet

```
100         self.bullet = pygame.Vector2(self.x, self.y - 12)
101
102     def hit_mushroom(self, cell):
103         self.mushrooms[cell] -= 1
104         if self.mushrooms[cell] <= 0:          # TASK 1: was "<= 1"
105             del self.mushrooms[cell]
106             self.flash.pop(cell, None)
107             self.score += 5
108         else:
109             self.flash[cell] = 0.1          # TASK 2: start the white flash
110
111     def split_chain(self, chain, index):
112         segment = chain[index]
113         self.chains.remove(chain)
114         left, right = chain[:index], chain[index + 1:]
115         self.chains.extend(part for part in (left, right) if part)
116         self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
117         self.score += 100 if index == 0 else 10
118         on_segment_hit(segment, self.score)
119
```

```

136         return
137
138     def update(self, dt, keys):
139         update_particles(dt) # TASK 3
140         self.flash = {c: t - dt for c, t in self.flash.items() if t - dt > 0} # TASK 2
141         if self.state != "play":
142             return
143         self.invulnerable = max(0.0, self.invulnerable - dt)
144         self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED * dt
145         self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED * dt
146         self.x = max(10, min(WIDTH - 10, self.x))
147         self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))
148         if keys[pygame.K_SPACE]:
149             self.fire()
150         self.update_bullet(dt)
151         self.timer += dt
152         effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
153         while self.timer >= effective_tick:
154             self.timer -= effective_tick
155             for chain in self.chains:
156                 for segment in chain:
157                     segment.step(self.mushrooms)
158         player_cell = (int(self.y // CELL), int(self.x // CELL))
159         if self.invulnerable <= 0 and any((s.row, s.col) == player_cell for c in self.chains for s in c):
160             self.lives -= 1
161             self.respawn()
162             if self.lives <= 0:
163                 self.state = "lose"
164         if not self.chains:
165             self.wave += 1
166             self.spawn_wave()
167

```

FULL CODE:

```

import math
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4

# TASK 3: spark particles, each is [x, y, vx, vy, life, max_life]
PARTICLES = []

# TASK 2: white flash when a mushroom has just been hit
def mushroom_color(hp, flashing=False):
    """Return an (r, g, b) colour for a mushroom, or None for the default."""
    if flashing:
        return (255, 255, 255)
    return None

```

```

# TASK 3: spark burst at the segment's last position
def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; spawns a spark effect."""
    x = segment.col * CELL + CELL / 2
    y = segment.row * CELL + CELL / 2
    for _ in range(14):
        angle = random.uniform(0, 2 * math.pi)
        speed = random.uniform(40, 160)
        PARTICLES.append([x, y, math.cos(angle) * speed, math.sin(angle) *
speed, 0.5, 0.5])

def update_particles(dt):
    for p in PARTICLES:
        p[0] += p[2] * dt
        p[1] += p[3] * dt
        p[4] -= dt
    PARTICLES[:] = [p for p in PARTICLES if p[4] > 0]

def draw_particles(screen):
    for x, y, _, _, life, max_life in PARTICLES:
        f = life / max_life
        color = (255, int(100 + 155 * f), int(60 * f))
        pygame.draw.circle(screen, color, (int(x), int(y)), max(1, int(3 * f)
+ 1))

# TASK 4: later waves are faster
def wave_speed_bonus(wave):
    """Return a tick-rate multiplier for centipede segments at the given
wave."""
    return 1 + 0.15 * (wave - 1)

class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:

```

```

        self.col = nxt

class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        self.flash = {}          # TASK 2: cell -> seconds of flash left
        PARTICLES.clear()        # TASK 3
        for _ in range(45):
            self.mushrooms[(random.randint(1, ZONE_TOP - 2), random.randint(0,
COLS - 1))] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5

    def spawn_wave(self):
        length = 10 + self.wave
        self.chains = [[Segment(0, COLS - 1 - i, -1) for i in range(length)]]
        self.timer = 0.0

    def fire(self):
        if self.bullet is None and self.state == "play":
            self.bullet = pygame.Vector2(self.x, self.y - 12)

    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if self.mushrooms[cell] <= 0:          # TASK 1: was "<= 1"
            del self.mushrooms[cell]
            self.flash.pop(cell, None)
            self.score += 5
        else:
            self.flash[cell] = 0.1            # TASK 2: start the white flash

    def split_chain(self, chain, index):
        segment = chain[index]
        self.chains.remove(chain)
        left, right = chain[:index], chain[index + 1:]
        self.chains.extend(part for part in (left, right) if part)
        self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP

```

```

        self.score += 100 if index == 0 else 10
        on_segment_hit(segment, self.score)

    def update_bullet(self, dt):
        if self.bullet is None:
            return
        self.bullet.y -= BULLET_SPEED * dt
        cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
        if self.bullet.y < 0:
            self.bullet = None
        elif cell in self.mushrooms:
            self.hit_mushroom(cell)
            self.bullet = None
        else:
            for chain in self.chains:
                for index, segment in enumerate(chain):
                    if (segment.row, segment.col) == cell:
                        self.split_chain(chain, index)
                        self.bullet = None
                        return

    def update(self, dt, keys):
        update_particles(dt)  # TASK
3
        self.flash = {c: t - dt for c, t in self.flash.items() if t - dt >
0} # TASK 2
        if self.state != "play":
            return
        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED
* dt
        self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED *
dt
        self.x = max(10, min(WIDTH - 10, self.x))
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))
        if keys[pygame.K_SPACE]:
            self.fire()
        self.update_bullet(dt)
        self.timer += dt
        effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
        while self.timer >= effective_tick:
            self.timer -= effective_tick
            for chain in self.chains:
                for segment in chain:
                    segment.step(self.mushrooms)
            player_cell = (int(self.y // CELL), int(self.x // CELL))
            if self.invulnerable <= 0 and any((s.row, s.col) == player_cell for c
in self.chains for s in c):

```

```

        self.lives -= 1
        self.respawn()
        if self.lives <= 0:
            self.state = "lose"
    if not self.chains:
        self.wave += 1
        self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))
        for (row, col), hp in self.mushrooms.items():
            color = mushroom_color(hp, (row, col) in self.flash) or (200 -
(MUSHROOM_HP - hp) * 40, 80, 170)
            center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
            pygame.draw.circle(screen, color, center, CELL // 2 - 1)
            pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3,
center[1], 6, CELL // 2 - 1))
            for chain in self.chains:
                for index, segment in enumerate(chain):
                    center = (segment.col * CELL + CELL // 2, segment.row * CELL +
CELL // 2)
                    pygame.draw.circle(screen, (240, 200, 60) if index == 0 else
(80, 220, 90), center, CELL // 2)
            draw_particles(screen) # TASK
3
        if self.bullet:
            pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1,
self.bullet.y - 6, 3, 10))
            if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
                pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y -
10), (self.x + 10, self.y + 10), (self.x - 10, self.y + 10)])
            hud = self.font.render(f"Score {self.score} Lives {self.lives} Wave
{self.wave} R = reset", True, (240, 240, 240))
            screen.blit(hud, (10, ROWS * CELL + 6))
            if self.state == "lose":
                label = self.font.render("GAME OVER - Press R", True, (255, 255,
120))
                screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT //
2)))

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:

```

```
dt = min(clock.tick(60) / 1000, 0.05)
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
        game.reset()
game.update(dt, pygame.key.get_pressed())
game.draw(screen)
pygame.display.flip()
pygame.quit()

if __name__ == "__main__":
    main()
```